MCA

unit - 6

Date
Page

↳ Design of frequency counters with display on LCD using 8051.

- Workflow:

i) Signal input:
Connect the input signal whose frequency you want to measure to one of the digital input pins of 8051.

ii) ~~counter~~ counter logic:
Configure one of the timers (like Timer 0 or Timer 1) to in counter mode.
Set the microcontroller to count the no. of rising and falling ages of the input signal.

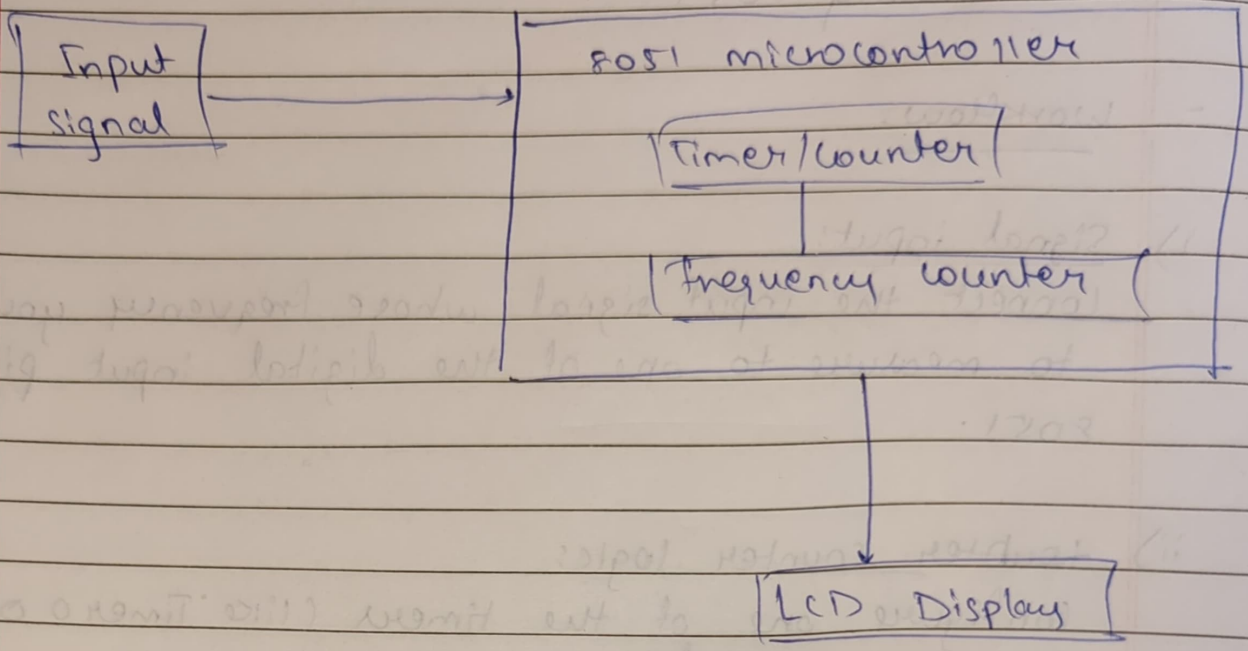
iii) Frequency calculation:
measure the time taken for certain pulses to occur. Then calculate the frequency by using formula.

$$\text{frequency} = \frac{\text{no. of Pulses}}{\text{Time}}$$

iv) LCD interfacing:
connect the LCD to microcontroller to display the calculated frequency.
use the appropriate GPIO pins for data & control lines.

v) Display update:
update the LCD display regularly with the calculated frequency.

- Diagram:



- code:

```
# include <8051.h>
# include <lcd.h>
sfr TO = 0x08;
sbit TRO = TCON^4;

void main() {
    unsigned long frequency;
    unsigned int timervalue;

  lcd_init();
  while(1) {
   TRO = 1;
  while (!TFO);
   TRO = 0;
   TFO = 0;
  timervalue = TH0 <<8, | TL0;
  frequency = 1 /(timer value * 12);
```

```
lcd_clear();
lcd_puts("Frequency :");
lcd_gotoxy(0,1);
```

↳ Design a water level monitoring system using 8051 microcontroller.

- Workflow:

i) Sensor interface:
Connect the water level sensors to the GPIO pins of the 8051 microcontroller.

ii) Analog to Digital conversion:
If using analog sensors, use ADC to convert the analog sensor readings to digital values.

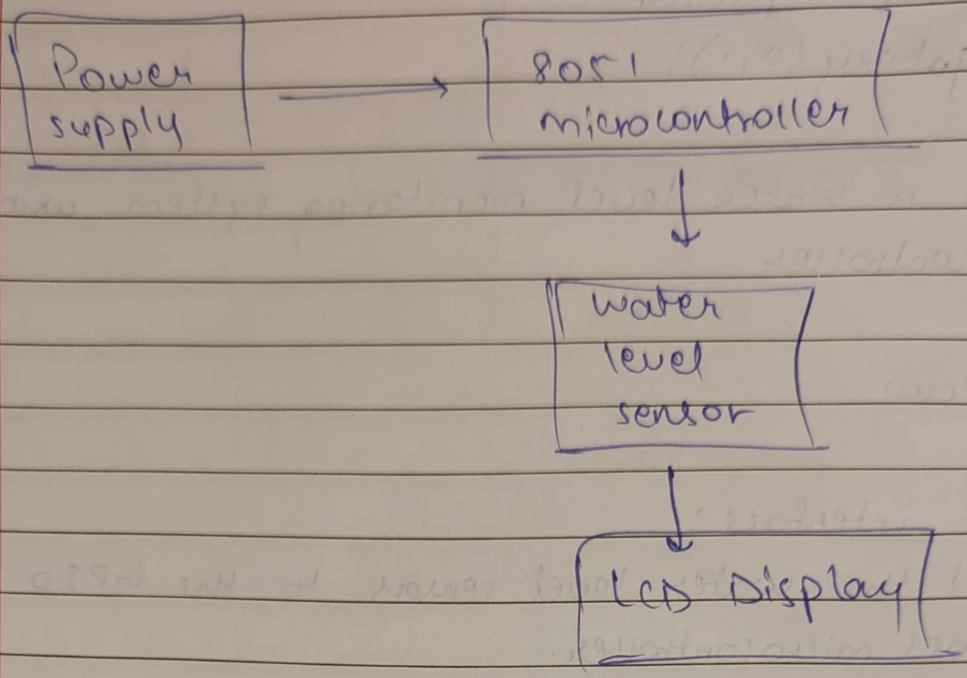
iii) Monitoring:
Read the sensor values periodically to determine the water level.

iv) Display:
Connect an output display (for e.g. LCD) to show the current water level status.

v) Alerting:
Implement a mechanism to trigger alerts (e.g. buzzers) when the water level crosses a certain threshold.

vi) Data logging:
Store the water level data for historical analysis if needed.

- Diagram:



- Code:

```
# include <8051.h>

sbit sensor = P1^0
sbit LED = P2^0
sbit Buzzer = P2^1

void main() {
   while(1){
   if (sensor==1){
     LED=1
     Buzzer=1;
   }
   else {
   LED=0
   Buzzer=0
   }
}
```

↳ Design of soil monitoring system using MSP430 microcontroller.

- Workflow:

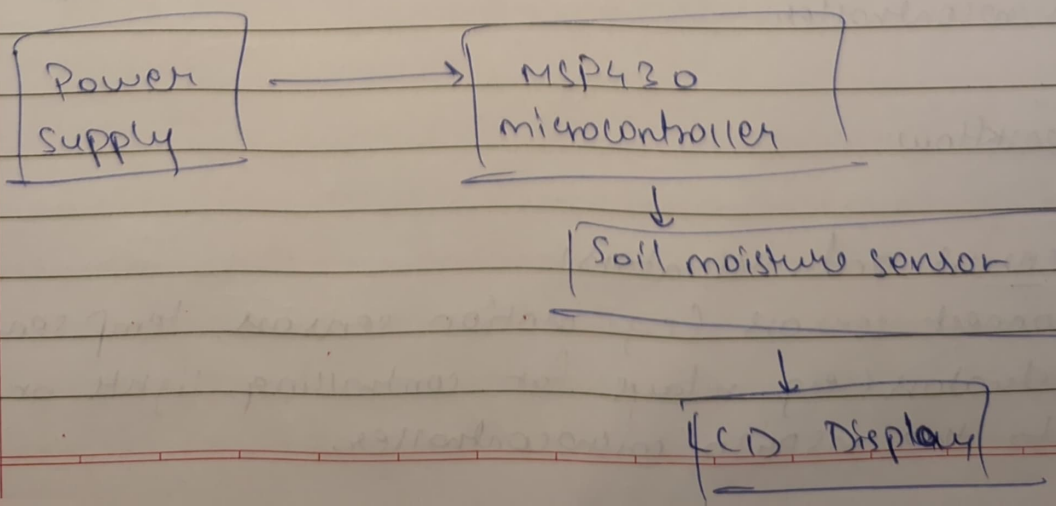
i) Sensor interfacing:
Connect the soil moisture sensor to ~~input pins of~~ the MSP430 microcontroller. It measures the soil moisture & sends the corresponding analog signal to the microcontroller.

ii) ADC:
microcontroller converts the analog signal to digital value using ADC.

iii) ~~Monitoring:~~ moisture level:
The microcontroller processes the digital value to determine the soil moisture level.

iv) Display:
connect the output display to show the soil moisture information.

- Diagram:



Code:

```
#include <msp430.h>

void initADC(){
  ADC10CTL0 = SREF_0 + ADC10SHT_3 + ADC10ON + ADC10CE;
  ADC10CTL1 = INCH_0;
  ADC10AE0 = BIT0;
  }

int main(void){
  WDTCTL = WDTPW + WDTHOLD;

  initADC();
  while(1);
  ADC10CTL0 |= ENC + ADC10SC;
  while(ADC10CTL1 & ADC10BUSY);
   int sensorvalue = ADC10MEM;

   __delay_cycles(1000)
   }
}
```

↳ Design a home automation system using MSP430 microcontroller.

- Workflow:

i) Sensors and Actuators:
connect sensors (e.g. motion sensors, temp sensors) & actuators (e.g. relays for controlling lights or appliances) to the MSP430 microcontroller.

ii) Monitoring:
continously monitor sensor inputs to detect events or changes. (e.g. turning on light when motion is detected)

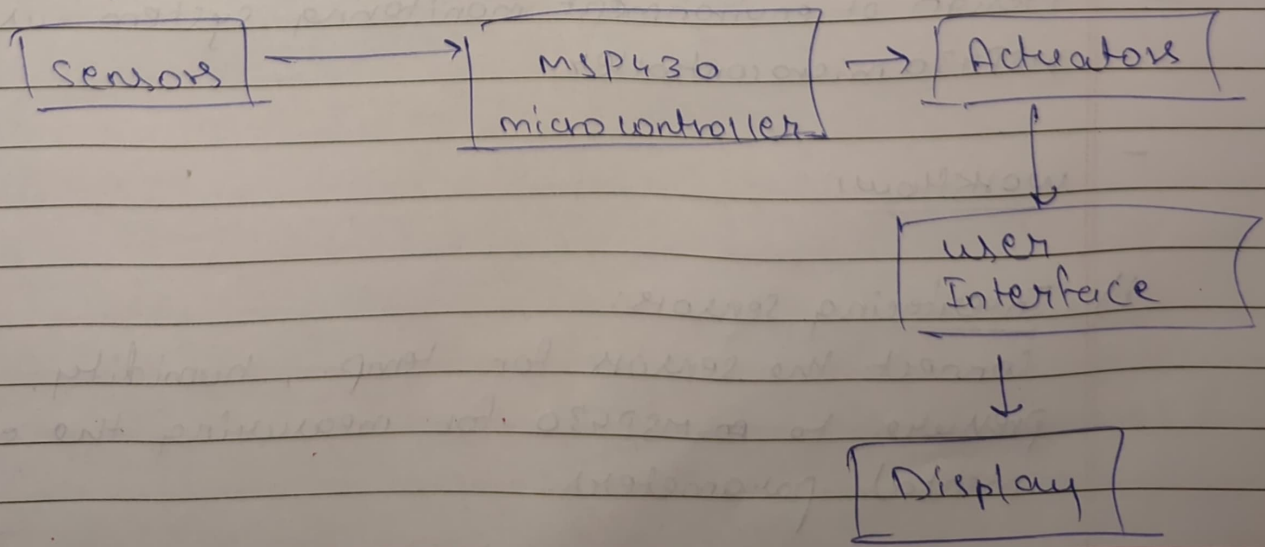
iii) Decision making:
Implement logic to make decisions based on sensor inputs.

iv) control Actuators:
Activate actuators as required (e.g. (e.g. turn on/off lights)

v) user interface:
Add a user interface, like a remote control, or a mobile app to interact with the system.

vi) Feedback and Display:
Provide feedback on the system's state & actions through display or indicators.

- Diagrams:



- Code:

```
#include <msp430.h>
#include <stdint.h>

#define MOTION_SENSOR BIT0
#define LIGHT_RELAY BIT1

void c){
 WDTCTL= WDTPW | WDTHOLD;
 P1DIR= LIGHT_RELAY;
 P1OUT &= ~LIGHT_RELAY;

 while(1){
 if (P1IN & MOTION_SENSOR){
    P1OUT |= LIGHT_RELAY;
    }
   else
   P1OUT &= ~LIGHT_RELAY;
  }
}
```

↳ Design of environment monitoring system using MSP430 microcontroller.

- Workflow:

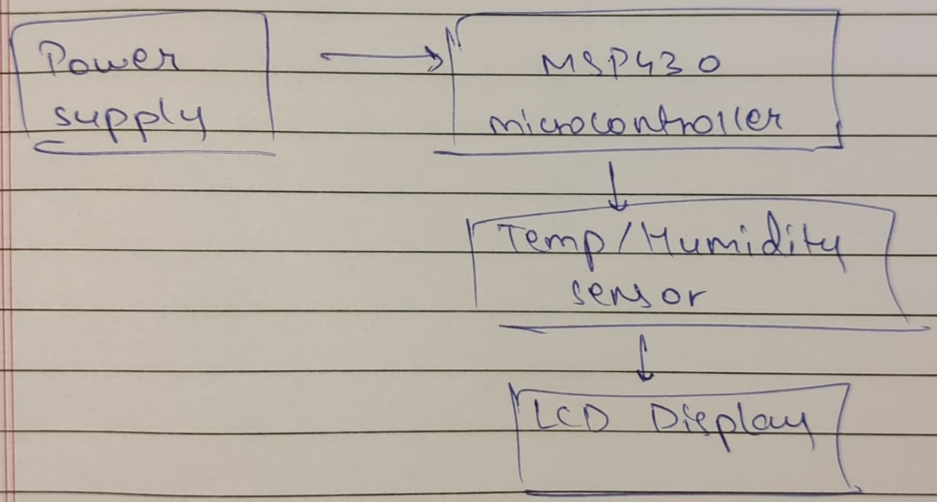
i) Interfacing sensors:
Connect the sensors for temp., humidity, atmospheric pressure to MSP430 for measuring the environmental parameters.

ii) Reading Sensor Data:
monitor the Data retrieved from the sensors.

iii) ADC:
Process the digital output obtained from ADC to determine the environmental parameters.

iv) Display:
Display the results on a display such as LCD.

- Diagram:



- code:

```
#include <msp430.h>
#include <dht11.h>
void main(void){
WDTCTL = WDTPW + WDTHOLD;
int DHT11();
while (1) {
int temperature, humidity;
if (readDHT11(&temperature, &humidity) == 0) {
}
delay_cycles (100000);
}
}
```